

Project 2: Algorithm Design & Analysis

(Divide & Conquer, Greedy, Dynamic Programming)

1. Objective

The goal of this project is to:

- Understand and apply **three major algorithmic paradigms**:
 - Divide & Conquer (D&C)
 - Greedy Algorithms
 - Dynamic Programming (DP)
- Implement solutions
- Analyze their efficiency
- Compare theoretical vs practical performance

2. Team & Tools

- Work in **teams of 2-3**
- Programming language: **C++ or Python only**
- You may use:
 - Standard libraries (e.g., vector, list, numpy)
- You may NOT use:
 - Built-in functions that directly solve the problem (e.g., no built-in sort for D&C problem)

3. REQUIRED PROBLEMS (NO CHOICE)

To avoid confusion, **all students must implement the SAME problems.**

A. Divide & Conquer (D&C)

Problem: Merge Sort

Requirements:

- Implement **Merge Sort from scratch**
- Do NOT use built-in sorting functions

Must include:

- Recursive splitting of array
- Merge function implementation

Test cases:

- Random array
- Already sorted array
- Reverse sorted array

B. Greedy Algorithm**Problem: Activity Selection Problem****Input:**

- A list of activities with:
 - Start time
 - Finish time

Requirements:

- Select maximum number of non-overlapping activities
- Must:
 - Sort activities by finish time
 - Apply greedy selection

Output:

- Selected activities
- Total number of activities selected

C. Dynamic Programming (DP)**Problem: 0/1 Knapsack****Input:**

- Weights array
- Values array
- Capacity of knapsack

Requirements:

- Implement DP solution using:
 - 2D table OR
 - Memoization

Output:

- Maximum value achievable

4. Implementation Requirements

Your program must:

Structure:

- Separate functions for each algorithm
- Example:
 - mergeSort()
 - activitySelection()
 - knapsack()

Code Quality:

- Proper indentation
- Meaningful variable names
- Comments explaining logic

5. Algorithm Explanation (MANDATORY)

For EACH problem, include in your report:

1. Explanation

- What is the problem?
- What is the approach?

2. Pseudocode

- Step-by-step logic

3. Why this paradigm works

- Why D&C here?
- Why Greedy works?
- Why DP is needed?

6. Complexity Analysis (MANDATORY)

For EACH algorithm:

Include:

- Time Complexity:
 - Best case
 - Worst case
- Space Complexity

Example expectation:

- Merge Sort $\rightarrow O(n \log n)$
- Activity Selection $\rightarrow O(n \log n)$
- Knapsack $\rightarrow O(n \times \text{capacity})$

7. Testing Requirements

You MUST test each algorithm with:

At least 3 test cases:

1. Small input
2. Medium input
3. Edge case

Examples:

- Empty input
- Single element
- Large input

8. Experimental Evaluation

For each algorithm:

Record:

- Input size
- Execution time

Present:

- Table OR simple observation

Example:

Input Size Time

10	X ms
100	X ms
1000	X ms

9. Report (PDF) – REQUIRED FORMAT

Your report must include:

Section 1: Introduction

- Brief overview of all 3 problems

Section 2: Divide & Conquer

- Explanation
- Pseudocode
- Complexity

Section 3: Greedy

- Explanation
- Pseudocode
- Complexity

Section 4: Dynamic Programming

- Explanation
- Pseudocode
- Complexity

Section 5: Results

- Test cases
- Observations

Section 6: Conclusion

- What you learned

10. Submission Requirements

Submit ALL of the following:

1. Code

- Single ZIP file OR GitHub link

2. Report (PDF)

3. Output Screenshots

- Show results of test cases

12. Important Rules

- No copying from internet or other teams
- You must understand your code
- If you cannot explain it → marks will be deducted

13. Common Mistakes (Avoid These)

- Using built-in sort for Merge Sort
- Using greedy logic for knapsack
- Writing brute-force instead of DP
- No complexity explanation
- No test cases

14. Summary Checklist (Students MUST follow)

Before submitting, confirm:

- Merge Sort implemented correctly
- Activity Selection implemented correctly
- 0/1 Knapsack using DP implemented
- Complexity analysis included
- At least 3 test cases per algorithm
- Report completed
- Code is clean and readable

Rubric:

Category	Criteria	Points
1. Merge Sort (Divide & Conquer)	Correct recursive splitting and merge implementation	5
	Produces correct sorted output for all test cases	3
	Proper base case and merge logic	2
	Subtotal	10
2. Activity Selection (Greedy)	Correct sorting by finish time	3
	Correct greedy selection logic	5
	Correct output (selected activities/count)	2
	Subtotal	10
3. 0/1 Knapsack (Dynamic Programming)	Correct DP table or memoization implementation	5
	Correct recurrence relation and transitions	3
	Produces correct maximum value	2
	Subtotal	10
4. Algorithm Explanation & Understanding	Clear explanation of all three problems (1 per algorithm)	9
	Justification of paradigm (why D&C, Greedy, DP)	6
	Subtotal	15
5. Complexity Analysis	Correct time complexity for all algorithms	9
	Correct space complexity for all algorithms	6
	Subtotal	15
6. Code Quality & Structure	Modular design (separate functions for each problem)	5
	Readability (naming, formatting, indentation)	5
	Comments explaining logic	5
	Subtotal	15
7. Testing & Output	At least 3 test cases per algorithm	5
	Includes edge cases	3
	Output is clear and correct	2
	Subtotal	10
8. Experimental Evaluation	Input size vs execution time recorded	3
	Basic interpretation of results	2

Category	Criteria	Points
	Subtotal	5
9. Report Quality	Follows required structure	2
	Includes pseudocode and explanations	2
	Clear, organized presentation	1
	Subtotal	5
	TOTAL	100

- Built-in sort used → **0 in Merge Sort section**
- Greedy logic incorrect → deduct up to **10 points**
- Knapsack not implemented using DP → **major deduction (up to full section)**
- Missing complexity → deduct from **Complexity Analysis section**
- Missing test cases → deduct from **Testing section**